

Learning $\mathbb{Z}_2$ Addition with a Quadratic Autoregressive Model

February 15, 2026

1 Introduction and Data

1.1 $\mathbb{Z}_2$ addition

The group $\mathbb{Z}_2 = \{0, 1\}$ under addition mod 2 defines the XOR operation:

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

1.2 Sequence structure

Sequences have length T (a multiple of 3). Positions come in *blocks of 3*: two random bits followed by their XOR.

$$\underbrace{a, b, a \oplus b}_{\text{block 1}} \underbrace{a, b, a \oplus b}_{\text{block 2}} \cdots \quad (1)$$

For example, with $T = 12$:

$$0, 0, 0 \mid 1, 1, 0 \mid 0, 1, 1 \mid 1, 0, 1.$$

1.3 True distribution

Each block is independent. Within a block the two “random” positions are uniform in $\{0, 1\}$ and the third is deterministic, giving exactly four valid triples per block:

$$\{(0, 0, 0), (0, 1, 1), (1, 0, 1), (1, 1, 0)\},$$

each with probability $1/4$. The full sequence probability is $(1/4)^{T/3}$.

1.4 Entropy

Per-position entropy contributions:

- Positions $0, 1 \pmod{3}$: each contributes $\ln 2$ nats (fully random).
- Position $2 \pmod{3}$: contributes 0 nats (deterministic given the first two).

Average per-position entropy:

$$H = \frac{2}{3} \ln 2 \approx 0.462 \text{ nats}. \quad (2)$$

This is the theoretical minimum loss any correct model can achieve.

2 Why a Linear Model Fails

Suppose we predict x_2 from x_0, x_1 using a single logistic unit:

$$P(x_2 = 1 \mid x_0, x_1) = \sigma(a + b_1 x_0 + b_2 x_1),$$

where $\sigma(z) = 1/(1 + e^{-z})$. We need this to match XOR:

x_0	x_1	target $P(x_2 = 1)$
0	0	0
0	1	1
1	0	1
1	1	0

From rows 1 and 2: increasing x_1 (with $x_0 = 0$) must *increase* the output, so $b_2 > 0$. From rows 3 and 4: increasing x_1 (with $x_0 = 1$) must *decrease* the output, so $b_2 < 0$. **Contradiction.** No choice of (a, b_1, b_2) works—XOR is not linearly separable.

The accompanying script `01_linear_fails.py` demonstrates this numerically.

3 Energy-Based vs. Autoregressive Quadratic Models

A natural idea is to use *pairwise (quadratic) interactions* to capture XOR. However, *where* the quadratic terms appear matters fundamentally.

3.1 Energy-based (Boltzmann) interpretation—doesn't work

Define a joint distribution via an energy function with pairwise terms:

$$\log P(\mathbf{x}) = a + \sum_i b_i x_i + \sum_{i < j} c_{ij} x_i x_j - \log Z.$$

Since $x_t \in \{0, 1\}$, the energy splits into terms involving x_t and terms that do not:

$$E(\mathbf{x}) = \underbrace{E_0(\mathbf{x}_{-t})}_{\text{no } x_t} + x_t \cdot \underbrace{(b_t + \sum_{i \neq t} c_{it} x_i)}_{\equiv \Delta}.$$

Computing the conditional:

$$P(x_t = 1 \mid \mathbf{x}_{-t}) = \frac{e^{E_0 + \Delta}/Z}{e^{E_0}/Z + e^{E_0 + \Delta}/Z} = \frac{e^{\Delta}}{1 + e^{\Delta}} = \sigma(\Delta).$$

Both Z and e^{E_0} cancel. Crucially, $\Delta = b_t + \sum_{i \neq t} c_{it} x_i$ is **linear** in the conditioning variables—the quadratic terms $c_{ij} x_i x_j$ where neither index is t ended up in E_0 and cancelled. So:

$$P(x_t = 1 \mid \text{rest}) = \sigma(b_t + \sum_{i \neq t} c_{it} x_i),$$

which is exactly the form proven insufficient for XOR in Section 2.

Key insight: A pairwise Boltzmann machine has quadratic interactions in the *joint* but only *linear* conditionals. Capturing XOR in the conditionals would require a **three-body** term $x_0 x_1 x_2$ in the energy—making it a higher-order Boltzmann machine.

3.2 Autoregressive interpretation—works

Instead of an energy-based joint, build the distribution autoregressively via the chain rule:

$$P(x_0, \dots, x_{T-1}) = \prod_{t=0}^{T-1} P(x_t \mid x_{<t}),$$

and put the quadratic features *directly inside each conditional*:

$$P(x_t = 1 \mid x_{<t}) = \sigma\left(b_t + \sum_{i<t} W_{ti} x_i + \sum_{i<j<t} C_{tij} x_i x_j\right). \quad (3)$$

Now the product $x_i x_j$ appears explicitly in the sigmoid argument—it does *not* cancel. This is a fundamentally different model family from the Boltzmann machine.

3.3 Concrete solution for position 2

We need $P(x_2 = 1 \mid x_0, x_1) = x_0 \oplus x_1$. The logit is:

$$\text{logit} = b_2 + W_{20} x_0 + W_{21} x_1 + C_{201} x_0 x_1.$$

Setting $b_2 = -L$, $W_{20} = W_{21} = 2L$, $C_{201} = -4L$ for large $L > 0$:

x_0	x_1	logit	$\sigma(\text{logit})$	target
0	0	$-L$	≈ 0	0
0	1	$-L + 2L = L$	≈ 1	1
1	0	$-L + 2L = L$	≈ 1	1
1	1	$-L + 4L - 4L = -L$	≈ 0	0

As $L \rightarrow \infty$ this converges to exact XOR. The quadratic term C_{201} is essential—it “bends” the decision boundary.

4 The Full Model

4.1 The conditional

Each conditional is parameterised as:

$$P(x_t = 1 \mid x_{<t}) = \sigma(\Delta_t), \quad \Delta_t = b_t + \sum_{i<t} W_{ti} x_i + \sum_{i<j<t} C_{tij} x_i x_j. \quad (4)$$

Since x_t is binary, $P(x_t = 0 \mid x_{<t}) = \sigma(-\Delta_t)$. Both cases are written compactly as $P(x_t \mid x_{<t}) = \sigma((2x_t - 1) \Delta_t)$, since $(2x_t - 1)$ flips the sign of Δ_t when $x_t = 0$.

4.2 Chain rule decomposition

The joint distribution is the product of all conditionals:

$$P(x_0, \dots, x_{T-1}) = \prod_{t=0}^{T-1} \sigma((2x_t - 1) \Delta_t). \quad (5)$$

There is **no partition function**—each factor is already normalised ($\sigma(\Delta) + \sigma(-\Delta) = 1$), so the product is automatically a valid distribution over $\{0, 1\}^T$.

4.3 Computational cost

Each entry C_{tij} appears in exactly one conditional (the one for position t , since the mask enforces $i < j < t$). Computing the full $\log P(\mathbf{x})$ touches each entry of C once—total work $O(T^3)$, the same as a single pass through C .

In practice we need $\log P(\mathbf{x})$ for:

- **Training:** compute $\log P(\mathbf{x})$ for a batch, take the gradient, update. This is what `binary_cross_entropy_with_logits` computes.
- **Sampling:** go left to right—compute Δ_0 , sample $x_0 \sim \text{Bernoulli}(\sigma(\Delta_0))$, compute Δ_1 (which depends on x_0), sample x_1 , and so on. Cost: $O(t^2)$ per position.

4.4 Parameters

Parameter	Shape	Constraint	Count
b	$(T,)$	—	T
W	(T, T)	lower-triangular ($i < t$)	$T(T-1)/2$
C	(T, T, T)	$i < j < t$	$O(T^3/6)$

For $T = 12$ (4 blocks), C has on the order of ~ 300 free parameters.

4.5 Loss

Negative log-likelihood per position, averaged over the batch:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}_{\text{data}} [x_t \log \sigma(\Delta_t) + (1 - x_t) \log(1 - \sigma(\Delta_t))]. \quad (6)$$

Theoretical minimum: $(2/3) \ln 2 \approx 0.462$ nats per position.

5 PyTorch Implementation

The accompanying script `02_train_full.py` implements the full model. Below we summarise the key components.

5.1 Data generation

Each training batch is generated on the fly (infinite-data regime, no overfitting). Block alignment is fixed: every sequence has its XOR triples at positions $(0, 1, 2)$, $(3, 4, 5)$, $(6, 7, 8)$, etc. The model does *not* need to discover where the triples are. However, the autoregressive conditionals cross block boundaries—when computing $P(x_5 | x_{<5})$, the model sees positions 0–4, not just the within-block context 3, 4. It must learn that the cross-block weights (e.g. $W_{5,0}$, $C_{5,0,1}$) should be zero.

5.2 Model class

The `QuadraticAutoregressive` module stores three parameter tensors (b , W , C) and two Boolean masks enforcing causality:

1. **Linear term:** W is masked to be strictly lower-triangular, then the linear contribution is computed as $\mathbf{x} W_{\text{masked}}^\top$.
2. **Quadratic term:** the outer product $\mathbf{x} \otimes \mathbf{x}$ of shape (B, T, T) is contracted against C_{masked} (with $i < j < t$) via `einsum`, yielding one scalar per (batch, position).
3. **Output:** bias + linear + quadratic gives the logits (pre-sigmoid values).

5.3 Training

Training uses Adam with learning rate 3×10^{-2} and batch size 512. Fresh data is sampled every step. Typical convergence for $T = 12$:

Step	Loss (nats)
0	0.6931
1000	0.4739
2000	0.4660
3000	0.4635
5000	0.4643

The loss converges close to the theoretical minimum of 0.462. Note that SGD loss is noisy (each step uses a fresh mini-batch), so the reported loss fluctuates.

5.4 Inspecting learned parameters

After training, the parameters exhibit clean structure:

- $b_t \approx -L$ at XOR positions, $b_t \approx 0$ at random positions.
- $W_{t,t-2} \approx W_{t,t-1} \approx 2L$ at XOR positions; all other entries near zero.
- $C_{t,t-2,t-1} \approx -4L$ at XOR positions; everything else near zero.

With $L \approx 6.5$, the model independently discovers the XOR solution for each block.

6 Sparse and Banded Versions

6.1 Parameter scaling of the full model

For an order- n interaction term, the source indices must satisfy $i_1 < i_2 < \dots < i_n < t$. The number of such tuples drawn from $\{0, \dots, t-1\}$ is $\binom{t}{n}$, so summing over all positions t :

Parameter	Total count	Growth
b_t	T	$O(T)$
W_{ti}	$T(T-1)/2$	$O(T^2)$
C_{tij}	$T(T-1)(T-2)/6$	$O(T^3)$
D_{tijk}	$T(T-1)(T-2)(T-3)/24$	$O(T^4)$
order- n	$\binom{T}{n+1}$	$O(T^{n+1})$

For $T = 100$, C alone has $\sim 160\text{k}$ parameters and D has ~ 4 million. This motivates restricting which entries are allowed.

6.2 Fully banded model—all indices close

The simplest restriction: position t can only interact with positions within a window of size k . For C_{tij} we require $t - i \leq k$ and $t - j \leq k$. Since $i < j < t$, the constraint $t - i \leq k$ is binding (it is the furthest from t). This forces all indices into a window of size k .

Parameter	Total	Growth
b_t	T	$O(T)$
W_{ti}	kT	$O(kT)$
C_{tij}	$k^2T/2$	$O(k^2T)$
D_{tijk}	$k^3T/6$	$O(k^3T)$
order- n	$\binom{k}{n}T$	$O(k^nT)$

The key feature: **linear in T** (times a constant depending on k and the order). For the $\mathbb{Z}_2$ data with $k = 3$, C has just ~ 300 parameters regardless of sequence length.

The accompanying script `03_train_banded.py` implements the banded model. With bandwidth 3, it achieves the same loss (≈ 0.462) as the full model, confirming purely local structure.

6.3 Long-range with local pairs

The fully banded model assumes all interactions are local: if position t depends on $x_i x_j$, then both i and j must be near t . Some data may have *long-range* dependencies where distant positions interact—but the interacting positions are near *each other*.

For C_{tij} the constraint becomes:

$$|i - j| \leq k, \quad i < j < t \quad (\text{causality}),$$

with no constraint on $t - i$ or $t - j$.

Parameter	Total	Growth
b_t	T	$O(T)$
W_{ti}	$T^2/2$	$O(T^2)$
C_{tij}	$kT^2/2$	$O(kT^2)$
D_{tijk}	$k^2T^2/2$	$O(k^2T^2)$
order- n	$O(k^{n-1}T^2)$	$O(k^{n-1}T^2)$

Note that W is unconstrained ($O(T^2)$) because a single source index has no pair to be “local” with. Banding only kicks in at order 2 and above.

6.4 Comparison of scaling

Scheme	W	C	D	order- n
Full	$O(T^2)$	$O(T^3)$	$O(T^4)$	$O(T^{n+1})$
Fully banded	$O(kT)$	$O(k^2T)$	$O(k^3T)$	$O(k^nT)$
Long-range, local pairs	$O(T^2)$	$O(kT^2)$	$O(k^2T^2)$	$O(k^{n-1}T^2)$

The tradeoff:

- **Fully banded** is linear in T —cheapest, but blind to long-range interactions.
- **Long-range, local pairs** is quadratic in T with one fewer power of k . It lets any position attend to distant correlated clusters, as long as the clusters are internally compact.
- **Full** is the most expressive but the most expensive, with $O(T^{n+1})$ growth.

For the $\mathbb{Z}_2$ data, fully banded with $k = 3$ suffices.

6.4.1 Sparse tensor implementation

For very large T , storing a dense (T, T, T) tensor is wasteful. PyTorch sparse COO tensors can materialise only the nonzero entries of C . For a banded model with bandwidth k this is $O(k^2 T)$ entries instead of $O(T^3)$. At tutorial scale ($T \leq 18$) the dense-with-mask approach is simpler and fast enough; the sparse approach pays off at larger T .

7 Exact Training (No Corpus)

The accompanying script `04_exact_training.py` implements this section.

7.1 Motivation

The $\mathbb{Z}_2$ distribution is simple enough that we can *enumerate* all 2^T possible sequences and compute the loss exactly, eliminating sampling noise. This is feasible for $T \leq 18$ ($2^{18} = 262,144$ sequences).

7.2 True distribution

Each sequence is either valid (all XOR constraints satisfied, probability $(1/4)^{T/3}$) or invalid (probability 0).

7.3 Exact expected loss

The expected negative log-likelihood under the true distribution is:

$$\mathcal{L}_{\text{exact}} = \sum_{\mathbf{x}} P_{\text{true}}(\mathbf{x}) [-\log P_{\text{model}}(\mathbf{x})], \quad (7)$$

where the sum runs over all 2^T binary sequences.

7.4 Training with exact loss

With $T = 9$ (3 blocks, $2^9 = 512$ sequences), exact training converges smoothly:

Step	Exact loss (nats)
0	0.693147
500	0.491853
1000	0.472786
1500	0.467549
2000	0.465349

No sampling noise—just clean gradient descent.

7.5 Comparison: exact vs. SGD

Property	SGD on samples	Exact training
Loss per step	Noisy (mini-batch)	Exact
Gradient	Stochastic estimate	True gradient
Cost per step	$O(B \cdot T^2)$	$O(2^T \cdot T^2)$
Scalability	Any T	$T \leq \sim 18$
Final solution	Same	Same

For small T , exact training is a useful debugging tool: if the model can represent the distribution, exact training will find it without sampling artefacts.

8 Results and Interpretation

8.1 Learned parameter structure

After training (either method), the parameters have clean structure:

- **Biases** (b): positions 0, 1, 3, 4, 6, 7, ... (random positions) have $b \approx 0$; positions 2, 5, 8, 11, ... (XOR positions) have $b \approx -L$ for large L .
- **Linear weights** (W): XOR position t depends on $t-2$ and $t-1$ with $W_{t,t-2} \approx W_{t,t-1} \approx 2L$. All other entries are near zero.
- **Quadratic weights** (C): the only large entries are $C_{t,t-2,t-1} \approx -4L$ at XOR positions.

The model discovers the **block-diagonal** structure entirely from data.

8.2 Verification checklist

1. **Loss convergence**: final loss ≈ 0.462 nats/position = $(2/3) \ln 2$. ✓
2. **XOR prediction accuracy**: sampling 1000 sequences and checking predictions at XOR positions gives accuracy ≈ 1.0 . ✓
3. **Banded model**: training with bandwidth 3 gives the same final loss. ✓
4. **Exact vs. SGD**: both converge to the same parameter values (up to optimisation noise). ✓

8.3 Takeaway

This tutorial demonstrates a key distinction in probabilistic modelling:

- *Where* you put interaction terms (in the energy vs. in the conditionals) determines the expressiveness of your model.
- A pairwise energy model has linear conditionals—insufficient for XOR.
- Putting quadratic features directly in the autoregressive conditionals gives the model the capacity to learn XOR.